

Taller Práctico: Docker + Nginx

Backend — Dockerización de una API y Reverse Proxy

Numero de integrantes: equipos de 3 estudiantes

Tecnologías: Docker, Docker Compose, Nginx, Node.js y Express

Resultado esperado: una API backend ejecutándose en Docker y accesible a través de Nginx.

1. Objetivos de aprendizaje

- Crear un Dockerfile para una aplicación backend.
- Construir y ejecutar una imagen Docker.
- Utilizar variables de entorno dentro de un contenedor.
- Crear una arquitectura con múltiples contenedores usando Docker Compose.
- Configurar Nginx como reverse proxy.
- Comprender la comunicación entre contenedores mediante una red Docker.
- Verificar el funcionamiento y diagnosticar errores básicos.

2. Escenario

Una empresa tiene una API REST desarrollada con Node.js y Express. La API actualmente funciona directamente en `http://localhost:3000`. El equipo de infraestructura solicita que la API sea ejecutada dentro de Docker, que Nginx sea el punto de entrada y que los usuarios no accedan directamente al contenedor de la API.

3. Parte 1 — Crear la API

Cree una aplicación Node.js utilizando Express. La API debe tener como mínimo:

Método	Endpoint	Respuesta
GET	/	Información de la API
GET	/health	Estado del servicio
GET	/api/products	Lista de productos
GET	/api/products/:id	Producto específico

La respuesta de `/health` debe tener una estructura similar a:

```
{
  "status": "ok",
  "service": "backend-api"
}
```

Condición: el puerto de la aplicación no debe estar escrito directamente en el código. Utilice la variable de entorno `PORT`.

`PORT=3000`

4. Parte 2 — Dockerizar el Backend

Cree un archivo Dockerfile. El contenedor debe:

1. Utilizar una imagen oficial de Node.js.
2. Establecer un directorio de trabajo.
3. Copiar primero los archivos necesarios para instalar dependencias.
4. Ejecutar `npm install`.
5. Copiar el código de la aplicación.
6. Exponer el puerto utilizado por la aplicación.
7. Ejecutar el servidor.

Estructura inicial:

```
backend/
├── src/
│   └── server.ts
├── package.json
├── package-lock.json
└── Dockerfile
```

Construya la imagen:

```
docker build -t backend-api .
```

Ejecute el contenedor:

```
docker run -d \
  --name backend-api \
  -p 3000:3000 \
  -e PORT=3000 \
  backend-api
```

Verifique el contenedor:

```
docker ps
```

Pruebe el endpoint:

```
http://localhost:3000/health
```

5. Parte 3 — Analizar el contenedor

```
docker logs backend-api
```

```
docker inspect backend-api
```

Identifique: ID del contenedor, imagen utilizada, puerto publicado, variables de entorno, redes y estado del contenedor.

Pregunta: ¿Cuál es la diferencia entre el puerto del contenedor y el puerto publicado en el host?

6. Parte 4 — Docker Compose

Ahora la aplicación debe ejecutarse mediante Docker Compose. La estructura debe quedar:

```
backend/
├── src/
│   └── server.ts
├── package.json
├── package-lock.json
├── Dockerfile
├── compose.yaml
└── nginx/
    └── nginx.conf
```

Cree dos servicios: api y nginx.

```
HOST
|
| :8080
v
+-----+
|  nginx  |
+-----+
|
| Docker Network
|
v
+-----+
|  api    |
| :3000   |
+-----+
```

7. Parte 5 — Configurar Nginx

Cree nginx/nginx.conf. Nginx debe escuchar en el puerto 8080 y enviar las peticiones al servicio api en el puerto 3000.

El punto clave es que Nginx no debe utilizar localhost para comunicarse con la API. Debe utilizar el nombre del servicio Docker:

```
http://api:3000
```

Pregunta: ¿Por qué `http://api:3000` funciona entre contenedores, mientras que `http://localhost:3000` no representa correctamente al contenedor api?

8. Parte 6 — Reverse Proxy

Configure Nginx para que:

- GET /health sea enviado a `http://api:3000/health`.
- GET /api/products sea enviado a `http://api:3000/api/products`.

El usuario únicamente debe conocer:

```
http://localhost:8080
```

9. Parte 7 — Eliminar el acceso directo al Backend

Una vez funcionando Nginx, modifique compose.yaml para que el servicio api no publique directamente el puerto 3000 al host.

El usuario no debería poder acceder mediante:

```
localhost:3000
```

La única entrada pública debe ser:

```
localhost:8080
```

Nginx debe seguir pudiendo acceder a:

```
api:3000
```

Pregunta: explique la diferencia entre ports y expose en Docker Compose.

10. Parte 8 — Pruebas

```
curl http://localhost:8080/
```

```
curl http://localhost:8080/health
```

```
curl http://localhost:8080/api/products
```

```
docker compose ps
```

```
docker compose logs nginx
```

```
docker compose logs api
```

11. Parte 9 — Diagnóstico de errores

Modifique temporalmente Nginx para utilizar http://localhost:3000 en lugar de http://api:3000.

```
docker compose down
```

```
docker compose up -d
```

```
curl http://localhost:8080/health
```

Responda:

8. ¿Qué error obtiene?
9. ¿Por qué ocurre?
10. ¿Por qué localhost no representa al contenedor api?
11. ¿Cómo solucionaría el problema?
12. ¿Qué comando utilizaría para verificar las redes Docker?

12. Parte 10 — Reto adicional

Agregue un tercer servicio llamado redis. No es necesario implementar todavía lógica de negocio con Redis. El objetivo es demostrar que los servicios pueden comunicarse utilizando nombres DNS proporcionados por Docker Compose.

El servicio debe ser referenciado por su nombre, por ejemplo: redis, y no mediante localhost.

13. Entregables

```
taller-docker-nginx/
├── src/
│   └── server.ts
├── nginx/
│   └── nginx.conf
├── Dockerfile
├── compose.yaml
├── package.json
├── package-lock.json
└── README.md
```

El README.md debe incluir:

- Descripción de la solución.
- Arquitectura implementada.
- Instrucciones para ejecutar el proyecto.
- Comandos Docker utilizados.
- Explicación de ports vs expose.
- Explicación de localhost vs nombre del servicio Docker.
- Evidencias de las pruebas realizadas.
- Explicación del error producido durante el ejercicio de troubleshooting.

14. Rúbrica de evaluación

Criterio	Puntaje
API REST funcionando correctamente	1,0
Dockerfile correcto y construcción de imagen	1,0
Docker Compose y comunicación entre contenedores	1,0
Configuración de Nginx como reverse proxy	1,0
Troubleshooting, README y explicación técnica	1,0
TOTAL	5,0

15. Sustentación del taller

Se escogerá un integrante al azar del equipo para explicar la arquitectura, funcionalidad del taller